《通信与网络》
Communications & Networks

TCP and Transmission Control

Referred Book: Computer Networks: A Systems Approach, (5th edition), by Peterson and Davie: 5 End-to-End Protocols; 6 Congestion Control and Resource Allocation

Thanks to some coursework material and slice from Princeton, MIT and Berkley.

1

Part 2

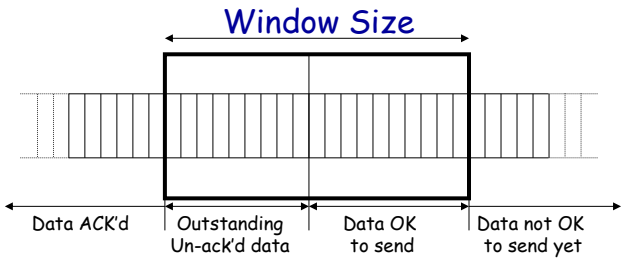
2

OPTIMIZING SLIDING WINDOW CONGESTION CONTROL

3

3

TCP Sliding Window



- Window is meaningful to the *sender*.
- Current window size is "advertised" by receiver (usually 4k – 8k Bytes when connection set-up).
- TCP's Retransmission policy is "Go Back N".

4

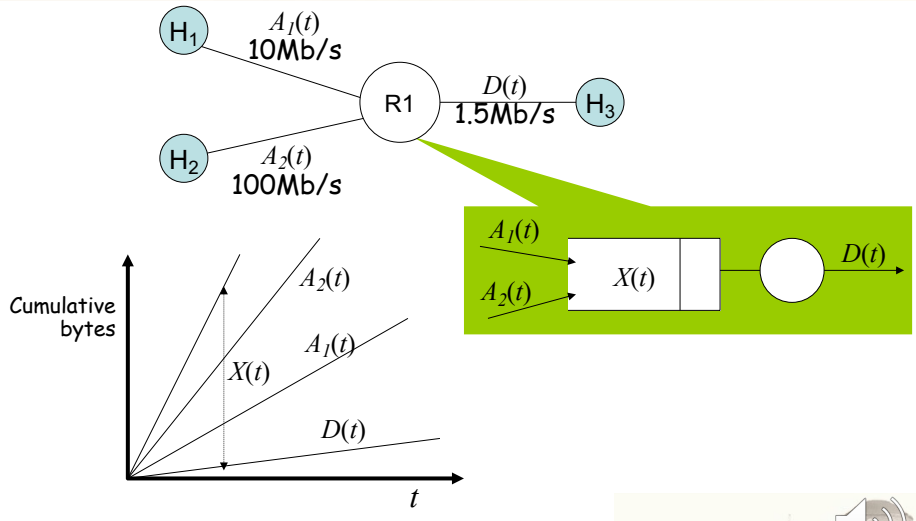
4

TCP Sliding Window

- How much data can a TCP sender have outstanding in the network?
- How much data should TCP retransmit when an error occurs? Just selectively repeat the missing data?
- How does the TCP sender avoid over-running the receiver's buffers?

5

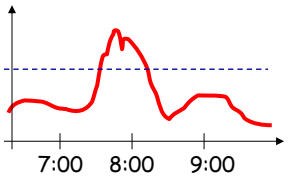
Congestion



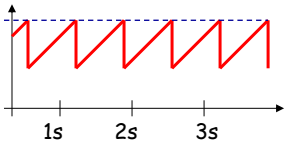
6

Time Scales of Congestion

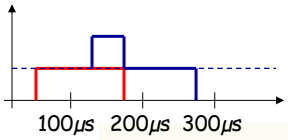
Too many users using a link during a peak hour



TCP flows filling up all available bandwidth



Two packets colliding at a router

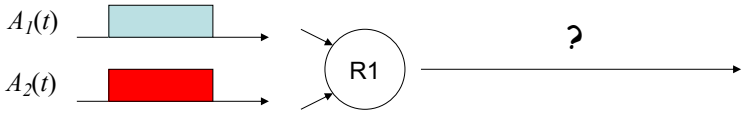


7

7

Dealing with Congestion

Example: two flows arriving at a router



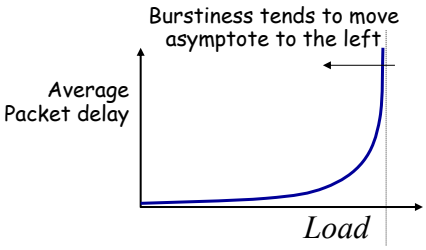
Strategy	
Drop one of the flows	
Buffer one flow until the other has departed, then send it	
Re-Schedule one of the two flows for a later time	
Ask both flows to reduce their rates	

8

8

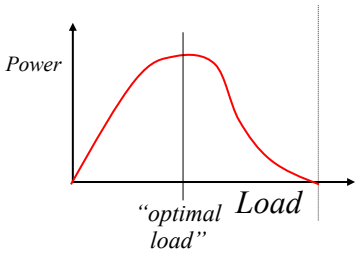
Congestion is Unavoidable

Typical behavior of queueing systems with random arrivals:



A simple metric of how well the network is performing:

$$Power = \frac{Load}{Delay}$$



9

9

TCP Congestion Control

- TCP implements host-based, feedback-based, window-based congestion control.
- TCP sources attempts to determine how much capacity is available
- TCP sends packets, then reacts to observable events (loss).

10

10

TCP Congestion Control

- TCP sources change the sending rate by modifying the window size:

  Window = min{Advertized window, Congestion Window}

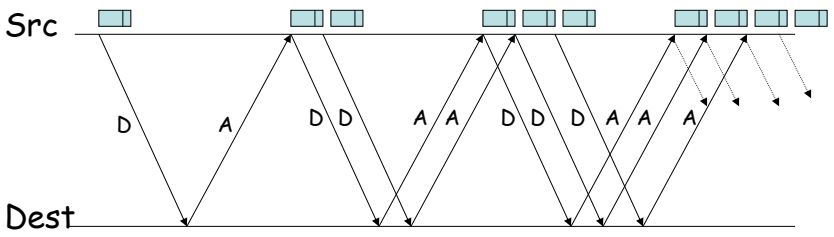
  Advertized window: Receiver; Congestion Window: Transmitter ("cwnd")

- In other words, send at the rate of the slowest component: network or receiver.
- "cwnd" follows additive increase/multiplicative decrease
  - On receipt of Ack: cwnd += 1
  - On packet loss (timeout): cwnd *= 0.5

11

11

Additive Increase



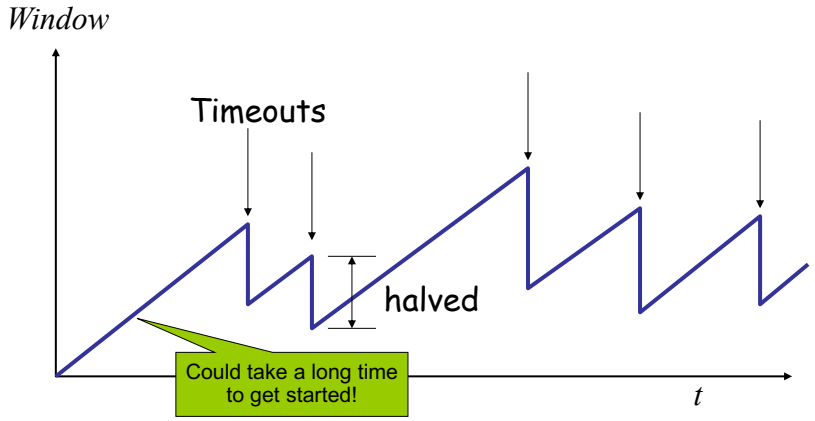
Actually, TCP uses bytes, not segments to count:
When ACK is received:

$$cwnd += MSS$$

12

12

Leads to the TCP "sawtooth"



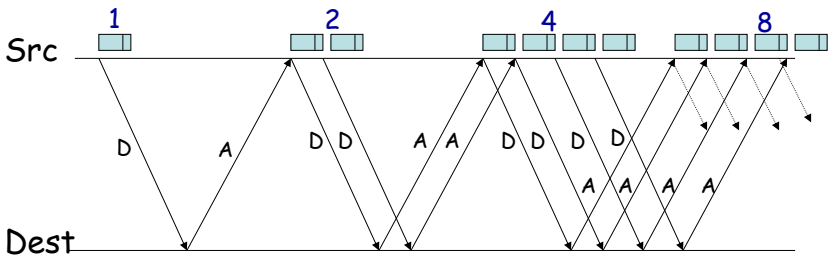
13

13

"Slow Start"

Designed to find the fair-share rate quickly at startup or if a connection has been halted (e.g. window dropped to zero, or window full, but ACK is lost).

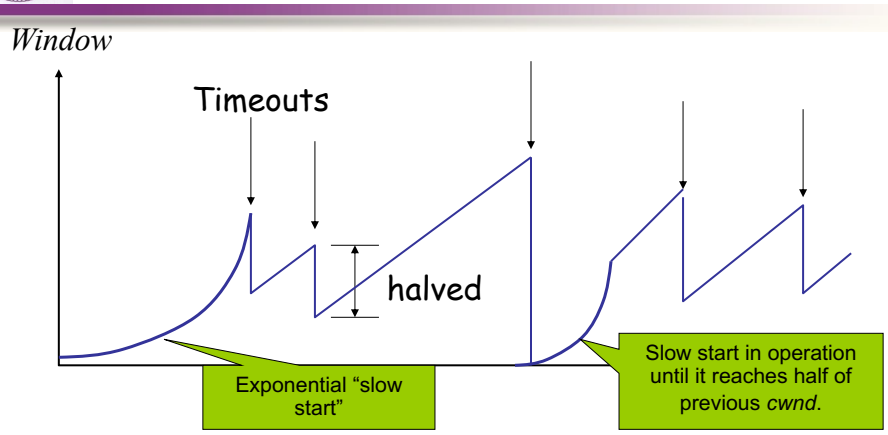
How it works: double cwnd for each round ACKs received.



14

14

Slow Start



Why is it called slow-start? Because TCP originally had no congestion control mechanism. The source would just start by sending a whole window's worth of data.

15

15

TCP Sending Rate

- What is the sending rate of TCP?
- Acknowledgement for sent packet is received after one RTT
- Amount of data sent until ACK is received is the current window size W
- Therefore sending rate is R = W/RTT

16

16

TCP and Buffers

- For TCP with a single flow over a network link with enough buffers, RTT and W are proportional to each other
- Therefore the sending rate $R = W/RTT$ is constant (and not a sawtooth)
- But experiments and theory suggest that with many flows:

$$R \propto \frac{1}{RTT\sqrt{p}}$$

Where: p is the drop probability. You'll see this in a problem set.

- TCP rate can be controlled in two ways:

1. Other layers: Buffering packets and increasing the RTT
2. Dropping packets to decrease TCP's window size

17

17

Congestion Control in the Internet

- Maximum window sizes of most TCP implementations by default are very small
  - Windows XP: 12 packets
  - Linux/Mac: 40 packets
- Often the buffer of a link is larger than the maximum window size of TCP
  - A typical DSL line has 200 packets worth of buffer
  - For a TCP session, the maximum number of packets outstanding is 40
  - The buffer can never fill up

18

18

Congestion Control

- Congestion is inevitable
- Congestion happens at different scales – from two individual packets colliding to too many users
- TCP Senders can detect congestion and reduce their sending rate by reducing the window size
- TCP modifies the rate according to “Additive Increase, Multiplicative Decrease (AIMD)”.
- To probe and find the initial rate, TCP uses a restart mechanism called “slow start”.
- Routers slow down TCP senders by buffering packets and thus increasing delay

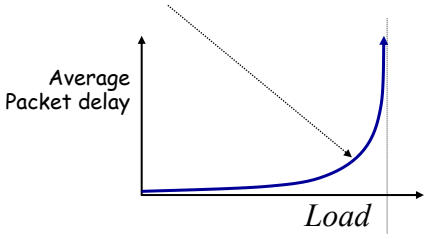
19

FROM CONGESTION CONTROL TO CONGESTION AVOIDANCE

20

Congestion Avoidance

- TCP reacts to congestion *after* it takes place. The data rate changes rapidly and the system is barely stable (or is even unstable).
- Can we *predict* when congestion is about to happen and avoid it? E.g. by detecting the knee of the curve.

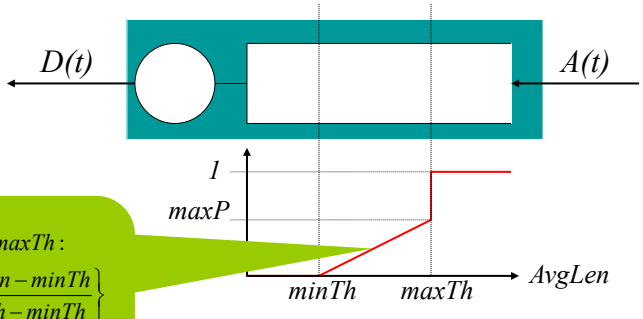


21

21

Router-based Congestion Avoidance
Random Early Detection (RED)

Routers implicitly notify sources by dropping packets.
RED drops packets at random, and as a function of the level of congestion.



If $minTh < AvgLen < maxTh$:

$$\hat{p}_{AvgLen} = maxP\left\{\frac{AvgLen - minTh}{maxTh - minTh}\right\}$$

$$\Pr(\text{Drop Packet}) = \frac{\hat{p}_{AvgLen}}{1 - count \times \hat{p}_{AvgLen}}$$

count counts how long we've been in $minTh < AvgLen < maxTh$ since we last dropped a packet. i.e. drops are spaced out in time, reducing likelihood of re-entering slow-start.

22

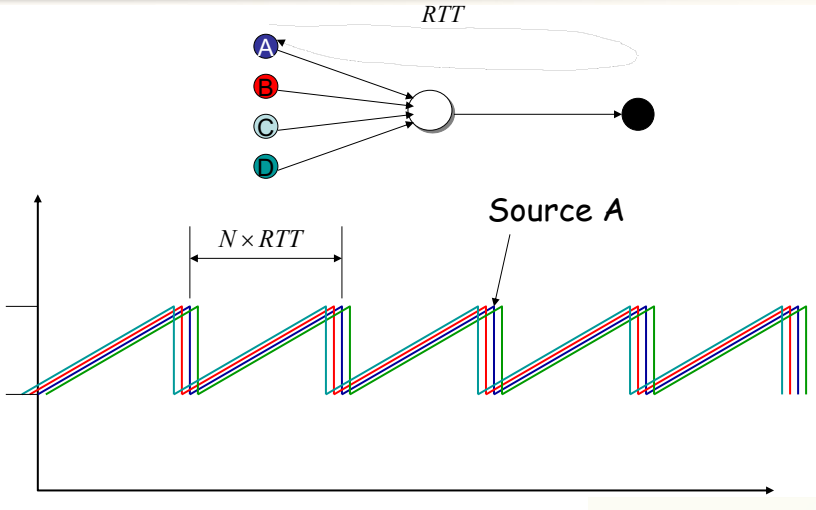
22

Properties of RED

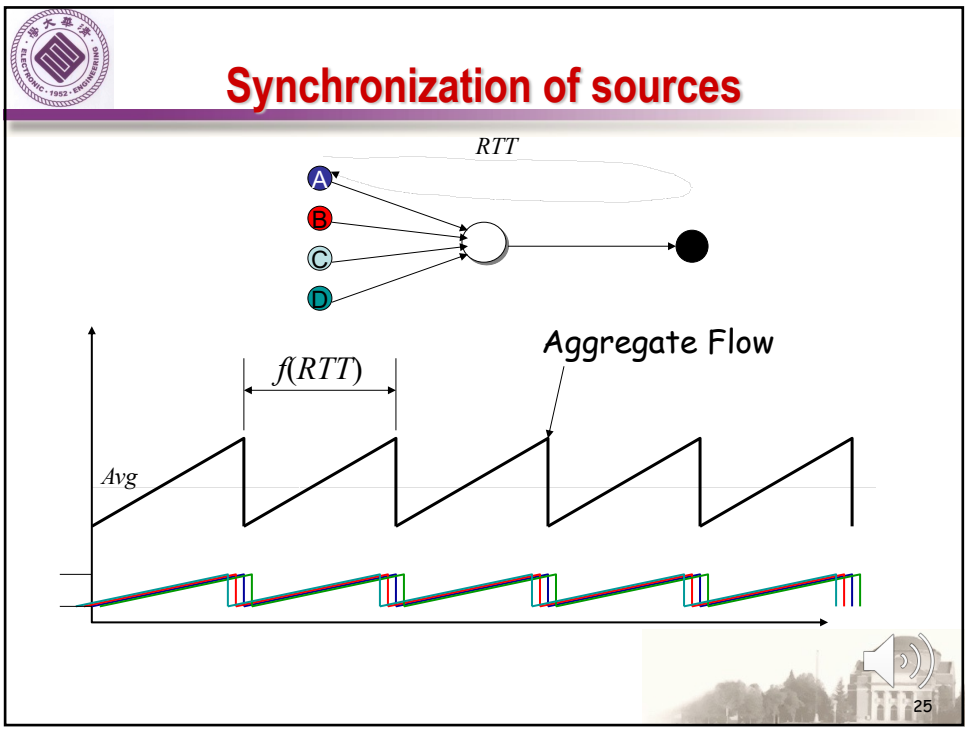
- Drops packets before queue is full, in the hope of reducing the rates of some flows.
- Drops packet for each flow *roughly* in proportion to its rate.
- Because it uses average queue length, RED is tolerant of bursts.
- Random drops hopefully desynchronize TCP sources.

23

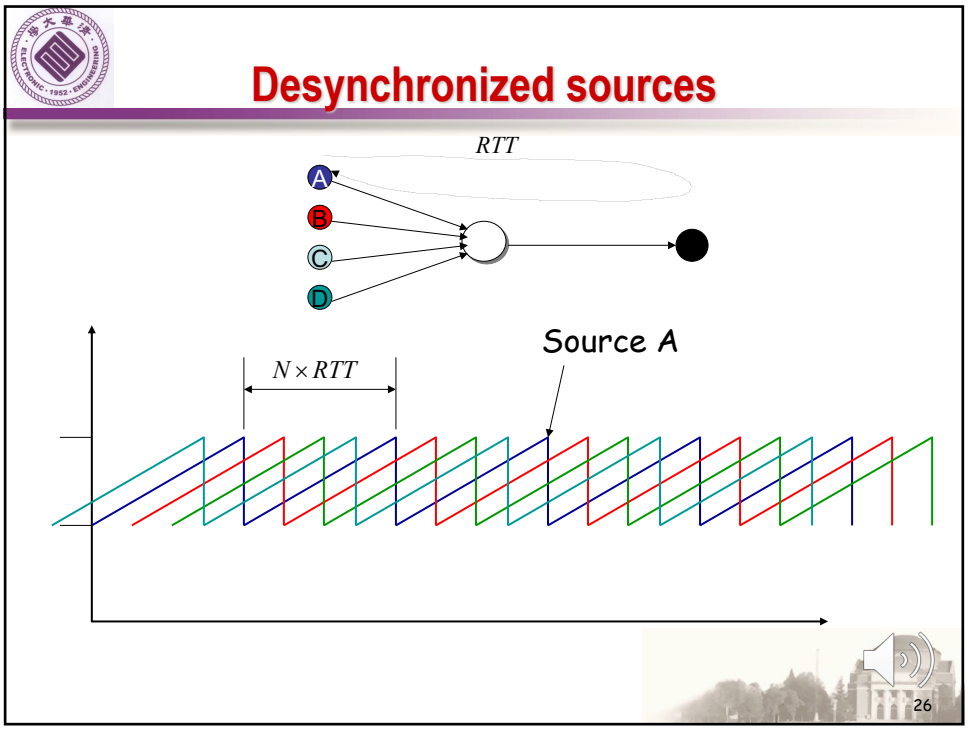
Synchronization of sources



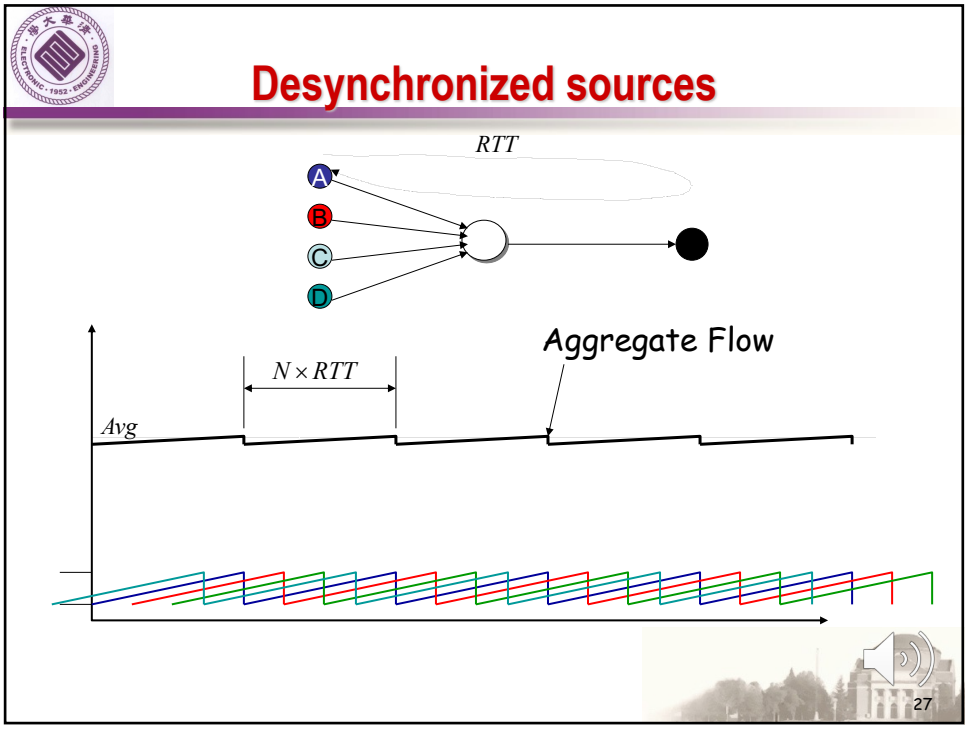
24


Synchronization of sources
RTT
A
B
C
D
Aggregate Flow
f(RTT)
Avg
25

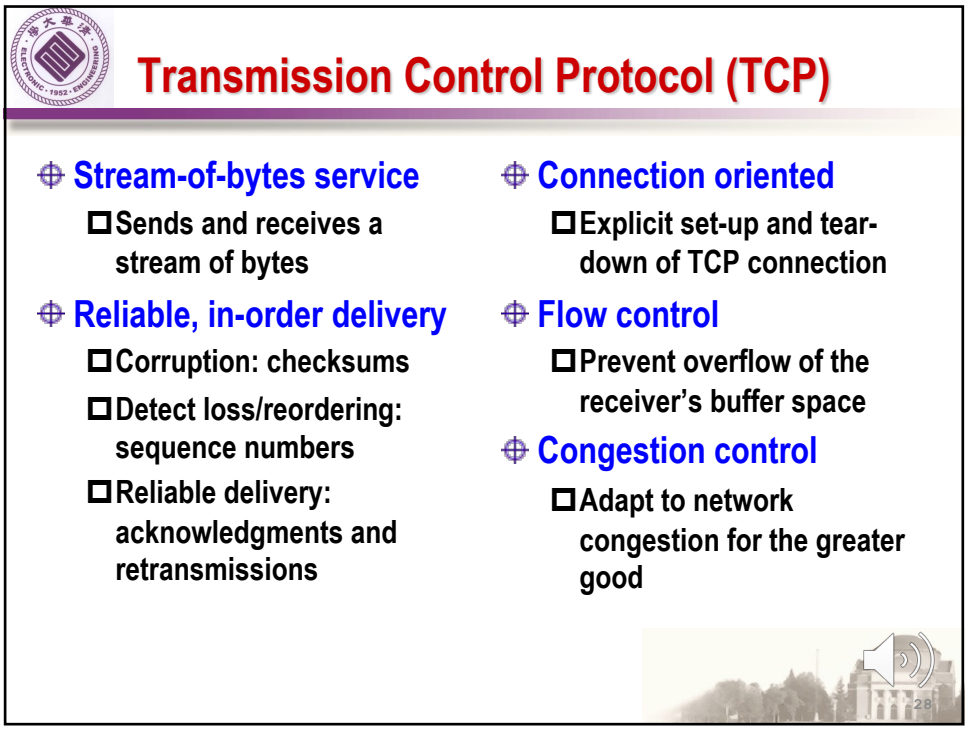
25


Desynchronized sources
RTT
A
B
C
D
Source A
N × RTT
26

26


Desynchronized sources
RTT
A
B
C
D
Aggregate Flow
N × RTT
Avg
27

27


Transmission Control Protocol (TCP)
Stream-of-bytes service
Sends and receives a stream of bytes
Reliable, in-order delivery
Corruption: checksums
Detect loss/reordering: sequence numbers
Reliable delivery: acknowledgments and retransmissions
Connection oriented
Explicit set-up and tear-down of TCP connection
Flow control
Prevent overflow of the receiver's buffer space
Congestion control
Adapt to network congestion for the greater good
28

28

Thanks !

29